

CSA0613

Design and Analysis of Algorithms

CO1_ASSESSMENT TOOL 1_Application-Based Question

Jaivant.S
(192311326)

Questions:

Q12. Application: Intelligent Decision Support Module:

An optimized decision-support algorithm reduces the problem size by half at each step and performs only constant work for evaluation and branching. Apply the substitution method to solve the recurrence $T(n)=T(n/2)+1$. Show the recursive expansion clearly and derive the final asymptotic complexity. Analyze how this improves efficiency for large inputs. Interpret the result using asymptotic notation.

Q35. Application: Video Compression Algorithm:

A video compression algorithm follows the recurrence: $T(n) = 8T(n/2) + n^3$. Use the Master Theorem to determine time complexity. Identify parameters and justify the case. Discuss how processing time increases with video resolution.

1.Application: Intelligent Decision Support Module:

Given Recurrence:

$$T(n) = T(n/2) + 1$$

Substitution Method (Expansion):

Expand the recurrence:

$$T(n) = T(n/2) + 1$$

$$= T(n/4) + 1 + 1$$

$$= T(n/8) + 1 + 1 + 1$$

$$= T(n/16) + 1 + 1 + 1 + 1$$

After **k** expansions,

$$T(n) = T(n/2^k) + k$$

The recursion stops when

$$n/2^k = 1$$

Therefore,

$$2^k = n$$

$$k = \log_2 n$$

Substituting,

$$T(n) = T(1) + \log_2 n$$

Since $T(1)$ is a constant,

$$T(n) = \Theta(\log n)$$

Efficiency Analysis:

- The problem size is reduced by half at every recursive call.
 - Each recursive call performs only **constant work (1)**.
 - The recursion continues for $\log_2 n$ levels.
 - Therefore, the total work grows logarithmically with the input size.
 - For large inputs, the algorithm remains highly efficient because the number of recursive calls increases very slowly compared to linear or quadratic algorithms.
-

Asymptotic Time Complexity:

$$\Theta(\log n)$$

Source Code:

```
1 def decision_support(n):
2     if n <= 1:
3         return 1
4     return decision_support(n // 2) + 1
5
6 n = int(input("Enter the problem size: "))
7
8 result = decision_support(n)
9
10 print("Decision Support Cost:", result)
11 print("Time Complexity:  $\Theta(\log n)$ ")
```

Output:

```
Output Clear  
Enter the problem size: 16  
Decision Support Cost: 5  
Time Complexity:  $\theta(\log n)$   
  
=== Code Execution Successful ===
```

2.Application: Video Compression Algorithm:

Given Recurrence:

$$T(n) = 8T(n/2) + n^3$$

Applying the Master Theorem:

The recurrence is of the form:

$$T(n) = aT(n/b) + f(n)$$

where,

- $a = 8$
- $b = 2$
- $f(n) = n^3$

Calculate:

$$n^{(\log_2 8)} = n^3$$

Compare:

- $f(n) = n^3$
- $n^{(\log_2 8)} = n^3$

Since,

$$f(n) = \Theta(n^{(\log_2 8)})$$

This satisfies Master Theorem Case 2, where

$$f(n) = \Theta(n^{(\log_b a)} \log^k n), k = 0$$

Therefore,

$$T(n) = \Theta(n^{(\log_b a)} \log^{(k+1)} n)$$

$$T(n) = \Theta(n^3 \log n)$$

Case Identification:

Master Theorem: Case 2

Because,

$$f(n) = \Theta(n^3) = \Theta(n^{(\log_2 8)})$$

the recurrence matches **Case 2** of the Master Theorem.

Processing Time Analysis:

- The video is divided into **8 equal subproblems** at each recursive step.
- Every recursive level performs **n^3** compression work.
- The recursion continues for **$\log_2 n$** levels.
- Therefore, the total processing time is:

$$n^3 \times \log n = n^3 \log n$$

As the video resolution increases, the amount of data grows significantly. Since each level requires cubic work and there are logarithmic recursive levels, the processing time increases rapidly. Thus, the algorithm is efficient for moderate input sizes but becomes computationally expensive for very high-resolution videos.

Asymptotic Time Complexity:

$$\Theta(n^3 \log n)$$

Source Code:

```
1 def video_compression(n):
2     if n <= 1:
3         return 1
4     return 8 * video_compression(n // 2) + (n ** 3)
5
6 n = int(input("Enter the video resolution size: "))
7
8 result = video_compression(n)
9
10 print("Estimated Compression Cost:", result)
11 print("Time Complexity:  $\Theta(n^3 \log n)$ ")
```

Output:

```
Enter the video resolution size: 8  
Estimated Compression Cost: 2048  
Time Complexity:  $\theta(n^3 \log n)$ 
```

```
=== Code Execution Successful ===
```